

AMRITA
VISHWA VIDYAPEETHAM
U N I V E R S I T Y
Established u/s 3 of UGC Act 1956
BENGALURU CAMPUS

Wireless Communication and Networks Laboratory

Subject Code:23ECE286
System Level Design
6th Sem-ECE C

Submitted By:

Name	Registration number
G. LAHARI REDDY	BL.EN.U4ECE23116
G. SOHAN	BL.EN.U4ECE23117
HARI SHANKAR	BL.EN.U4ECE23118

Submitted To:

Name	Dr. Sumathi S Dr. Harshit Srivastava
Department	ECE department

1)

Aim :

Write a C/C++ program to implement the TCP protocol.

Tool used : terminal

Theory :

Transmission Control Protocol (TCP) is a transport layer communication protocol that enables reliable data transfer between devices over a network by establishing a connection before communication begins. It is a connection-oriented protocol, meaning a connection is formed using a three-way handshake before any data is transmitted. TCP ensures that data is delivered accurately, in the correct order, and without loss by using acknowledgments, error detection, and retransmission mechanisms. In a TCP-based client–server model, the server listens for incoming connection requests on a specific port, while the client initiates a connection using the server's address and port number. Once the connection is established, both client and server can send and receive data simultaneously. TCP is widely used in applications such as web browsing, file transfer, and email services where reliable and ordered communication is essential.

Procedure :

- Open Command Prompt on your system
- Create a project folder using mkdir tcp_project
- Navigate into the folder using cd tcp_project
- Create server file using notepad tcp_chat_server.c
- Write TCP server socket code and save the file
- Create client file using notepad tcp_chat_client.c
- Write TCP client socket code and save the file
- Compile server using gcc tcp_chat_server.c -o server -lws2_32
- Compile client using gcc tcp_chat_client.c -o client -lws2_32
- Run server (server) in one terminal
- Open another terminal and run client (client)
- Send messages from client and observe communication between multiple clients through the server

Server code :

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <pthread.h>
#define BACKLOG        50
#define RECV_BUF_SIZE  4096
#define MAX_CLIENTS    100
typedef struct {
   int    socket;
   char   ip[INET_ADDRSTRLEN];
   int    port;
   int    active;
} ClientSlot;
static ClientSlot  g_clients[MAX_CLIENTS];
static pthread_mutex_t g_lock = PTHREAD_MUTEX_INITIALIZER;
static void
broadcast(const char *msg, int len, int exclude)
{
   pthread_mutex_lock(&g_lock);
   for (int i = 0; i < MAX_CLIENTS; i++) {
      if (g_clients[i].active && g_clients[i].socket != exclude) {
         send(g_clients[i].socket, msg, len, 0);
      }
   }
   pthread_mutex_unlock(&g_lock);
}
static int
add_client(int fd, const char *ip, int port)
{
   pthread_mutex_lock(&g_lock);
   for (int i = 0; i < MAX_CLIENTS; i++) {
      if (!g_clients[i].active) {
         g_clients[i].socket = fd;
         g_clients[i].port   = port;
```

```
            g_clients[i].active = 1;
            strncpy(g_clients[i].ip, ip, INET_ADDRSTRLEN - 1);
            pthread_mutex_unlock(&g_lock);
            return i;
        }
    }
    pthread_mutex_unlock(&g_lock);
    return -1;
}
static void
remove_client(int idx)
{
    pthread_mutex_lock(&g_lock);
    g_clients[idx].active = 0;
    pthread_mutex_unlock(&g_lock);
}
static int
active_count(void)
{
    int n = 0;
    pthread_mutex_lock(&g_lock);
    for (int i = 0; i < MAX_CLIENTS; i++)
        if (g_clients[i].active) n++;
    pthread_mutex_unlock(&g_lock);
    return n;
}
typedef struct {
    int    socket;
    char   ip[INET_ADDRSTRLEN];
    int    port;
} ClientCtx;
static void*
client_thread(void* arg)
{
    ClientCtx *ctx = (ClientCtx *)arg;
    int        fd  = ctx->socket;
    char       ip[INET_ADDRSTRLEN];
    int        port;
    strncpy(ip, ctx->ip, sizeof(ip) - 1);
    ip[sizeof(ip) - 1] = '\0';
    port = ctx->port;
    free(ctx);
```

```
    int idx = add_client(fd, ip, port);
    if (idx < 0) {
        printf("[!] Client table full, rejecting %s:%d\n", ip, port);
        close(fd);
        return NULL;
    }
    printf("[+] %s:%d connected  (active: %d)\n", ip, port,
active_count());
    char username[64] = "";
    int has_username = 0;
    char buf[RECV_BUF_SIZE];
    int  total = 0;
    int  has_announced_join = 0;
    int  is_first_msg = 1;
    while (total < (int)sizeof(buf) - 1) {
        int n = recv(fd, buf + total, sizeof(buf) - 1 - total, 0);
        if (n <= 0) break;
        total += n;
        char *start = buf;
        char *nl;
        while ((nl = (char *)memchr(start, '\n', buf + total - start)) !=
NULL) {
            int line_len = (int)(nl - start) + 1;
            if (is_first_msg) {
                is_first_msg = 0;
                if (line_len >= 5 && strncmp(start, "HELO:", 5) == 0) {
                    int ulen = line_len - 6; /* exclude HELO: and \n */
                    if (ulen > 0 && ulen < (int)sizeof(username)) {
                        strncpy(username, start + 5, ulen);
                        username[ulen] = '\0';
                        has_username = 1;
                    }
                    char notice[128];
                    int nlen;
                    if (has_username) {
                        nlen = snprintf(notice, sizeof(notice), "[Server] %s
(%s:%d) joined the chat\n", username, ip, port);
                    } else {
                        nlen = snprintf(notice, sizeof(notice), "[Server] %s:%d
joined the chat\n", ip, port);
                    }
                    broadcast(notice, nlen, fd);
                    has_announced_join = 1;
```

```
                start += line_len;
                continue;
            } else {
                char notice[128];
                int nlen = snprintf(notice, sizeof(notice), "[Server] %s:%d
joined the chat\n", ip, port);
                broadcast(notice, nlen, fd);
                has_announced_join = 1;
            }
        }
        char prefix[128];
        if (has_username) {
            snprintf(prefix, sizeof(prefix), "%s", username);
        } else {
            snprintf(prefix, sizeof(prefix), "%s:%d", ip, port);
        }
        char msg[RECV_BUF_SIZE + 64];
        int  msg_len = snprintf(msg, sizeof(msg),
                                "%s> %.*s",
                                prefix, line_len, start);
        if (msg_len > 0 && msg[msg_len - 1] != '\n') {
            msg[msg_len++] = '\n';
            msg[msg_len]   = '\0';
        }
        printf("[<] %s", msg);
        broadcast(msg, msg_len, fd);
        start += line_len;
    }
    int leftover = (int)(buf + total - start);
    if (leftover > 0)
        memmove(buf, start, leftover);
    total = leftover;
}
if (!has_announced_join) {
    char notice[128];
    int nlen = snprintf(notice, sizeof(notice), "[Server] %s:%d joined
the chat\n", ip, port);
    broadcast(notice, nlen, fd);
}
char notice[128];
int nlen;
if (has_username) {
```

```
        nlen = snprintf(notice, sizeof(notice), "[Server] %s (%s:%d) left
the chat\n", username, ip, port);
    } else {
        nlen = snprintf(notice, sizeof(notice), "[Server] %s:%d left the
chat\n", ip, port);
    }
    broadcast(notice, nlen, fd);
    remove_client(idx);
    printf("[-] %s:%d disconnected (active: %d)\n", ip, port,
active_count());
    close(fd);
    return NULL;
}
int main(int argc, char *argv[])
{
    if (argc < 2) {
        fprintf(stderr, "Usage: %s <port>\n", argv[0]);
        return 1;
    }
    memset(g_clients, 0, sizeof(g_clients));
    int port = atoi(argv[1]);
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd < 0) {
        perror("socket");
        return 1;
    }
    int yes = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &yes,
sizeof(yes));
    struct sockaddr_in addr;
    memset(&addr, 0, sizeof(addr));
    addr.sin_family      = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port        = htons((uint16_t)port);
    if (bind(server_fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        perror("bind");
        close(server_fd);
        return 1;
    }
    if (listen(server_fd, BACKLOG) < 0) {
        perror("listen");
        close(server_fd);
        return 1;
```

```
    }
    printf("[*] Chat server listening on port %d (max %d clients)\n\n",
        port, MAX_CLIENTS);
    for (;;) {
      struct sockaddr_in client_addr;
      socklen_t client_len = sizeof(client_addr);
      int client_fd = accept(server_fd,
                (struct sockaddr *)&client_addr,
                &client_len);
      if (client_fd < 0) {
        perror("accept");
        continue;
      }
      ClientCtx *ctx = (ClientCtx *)malloc(sizeof(ClientCtx));
      if (!ctx) { close(client_fd); continue; }

      ctx->socket = client_fd;
      ctx->port   = ntohs(client_addr.sin_port);
      inet_ntop(AF_INET, &client_addr.sin_addr, ctx->ip, sizeof(ctx-
>ip));
      pthread_t t;
      if (pthread_create(&t, NULL, client_thread, ctx) != 0) {
        fprintf(stderr, "pthread_create failed\n");
        free(ctx);
        close(client_fd);
        continue;
      }
      pthread_detach(t);
    }
    close(server_fd);
    return 0;
}
```

Client code :

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
```

```
#include <arpa/inet.h>
#include <pthread.h>
#define RECV_BUF_SIZE  4096
static volatile int g_running = 1;
static void*
recv_thread(void* arg)
{
    int fd = *(int *)arg;
    char buf[RECV_BUF_SIZE];
    int total = 0;
    while (g_running) {
        int n = recv(fd, buf + total, sizeof(buf) - 1 - total, 0);
        if (n <= 0) {
            if (g_running)
                printf("\n[*] Server closed the connection\n");
            g_running = 0;
            break;
        }
        total += n;
        char *start = buf;
        char *nl;
        while ((nl = (char *)memchr(start, '\n', buf + total - start)) != NULL) {
            int line_len = (int)(nl - start) + 1;
            printf("%.*s", line_len, start);
            fflush(stdout);
            start += line_len;
        }
        int leftover = (int)(buf + total - start);
        if (leftover > 0)
            memmove(buf, start, leftover);
        total = leftover;
    }
    return NULL;
}
int main(int argc, char *argv[])
{
    if (argc < 3) {
        fprintf(stderr, "Usage: %s <host> <port> [username]\n", argv[0]);
        return 1;
    }
    const char *host = argv[1];
    int         port = atoi(argv[2]);
    const char *username = argc > 3 ? argv[3] : "";
```

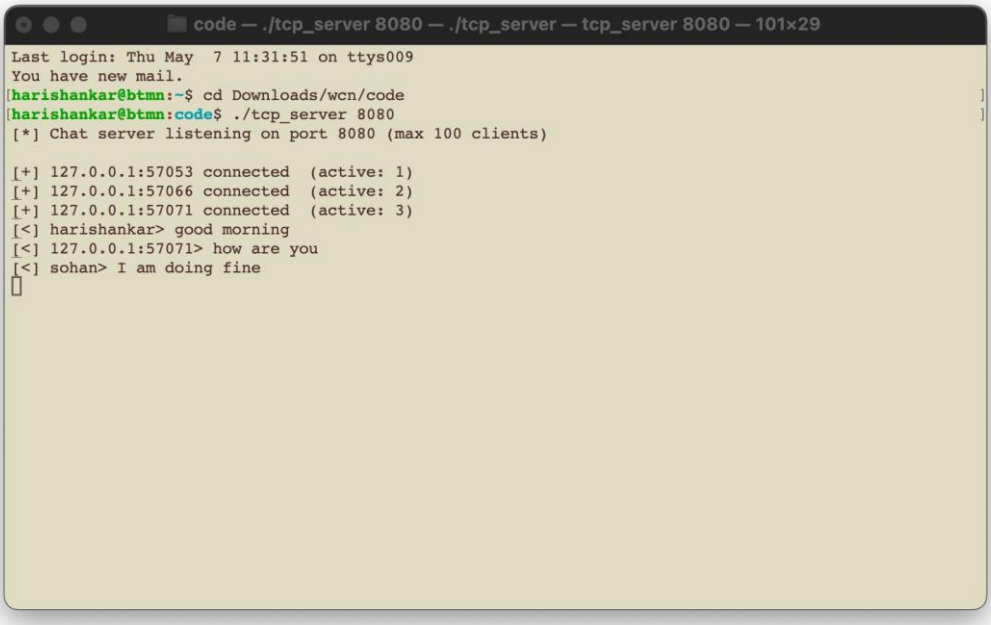
```
    char port_str[8];
    snprintf(port_str, sizeof(port_str), "%d", port);
    struct addrinfo hints, *res;
    memset(&hints, 0, sizeof(hints));
    hints.ai_family   = AF_INET;
    hints.ai_socktype = SOCK_STREAM;
    if (getaddrinfo(host, port_str, &hints, &res) != 0) {
        fprintf(stderr, "Cannot resolve '%s'\n", host);
        return 1;
    }
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) {
        perror("socket");
        freeaddrinfo(res);
        return 1;
    }
    printf("[*] Connecting to %s:%d ...\n", host, port);
    if (connect(fd, res->ai_addr, res->ai_addrlen) < 0) {
        perror("connect");
        close(fd);
        freeaddrinfo(res);
        return 1;
    }
    freeaddrinfo(res);
    char helo[256];
    snprintf(helo, sizeof(helo), "HELO:%s\n", username);
    send(fd, helo, strlen(helo), 0);
    printf("[*] Connected! Type messages and press Enter. Type 'quit' to
exit.\n\n");

    pthread_t t;
    if (pthread_create(&t, NULL, recv_thread, &fd) != 0) {
        fprintf(stderr, "pthread_create failed\n");
        close(fd);
        return 1;
    }
    char buf[1024];
    while (g_running) {
        if (!fgets(buf, sizeof(buf) - 2, stdin)) break;
        int len = (int)strlen(buf);
        while (len > 0 && (buf[len-1] == '\n' || buf[len-1] == '\r'))
            len--;
        buf[len] = '\0';
```

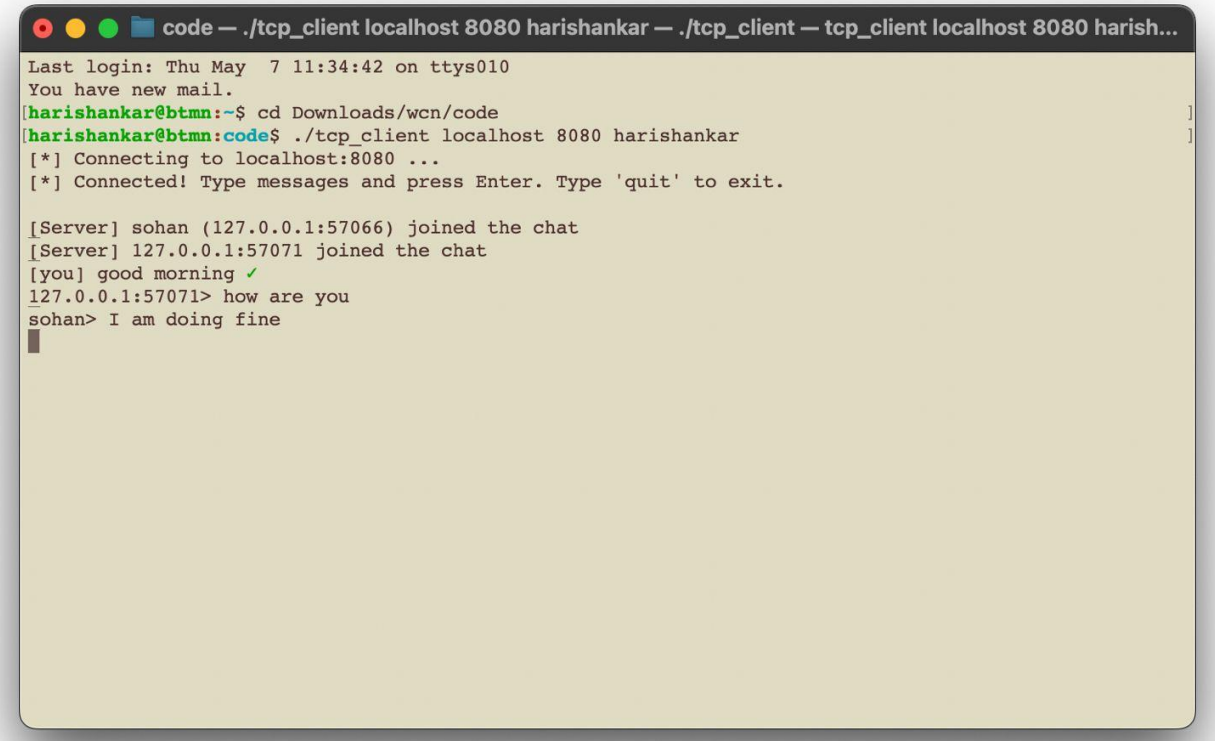
```
        if (strcmp(buf, "quit") == 0 || strcmp(buf, "exit") == 0) {
            printf("[*] Disconnecting...\n");
            break;
        }
        buf[len]     = '\n';
        buf[len + 1] = '\0';
        len++;
        int sent = 0;
        while (sent < len) {
            int n = send(fd, buf + sent, len - sent, 0);
            if (n < 0) { g_running = 0; break; }
            sent += n;
        }
        if (g_running) {
            printf("\033[1A\r\033[2K[you] %.*s \033[32m✓\033[0m\n", len - 1,
buf);
        }
    }
    g_running = 0;
    shutdown(fd, SHUT_RDWR);
    close(fd);
    pthread_join(t, NULL);
    printf("[*] Connection closed.\n");
    return 0;
}
```

Results :

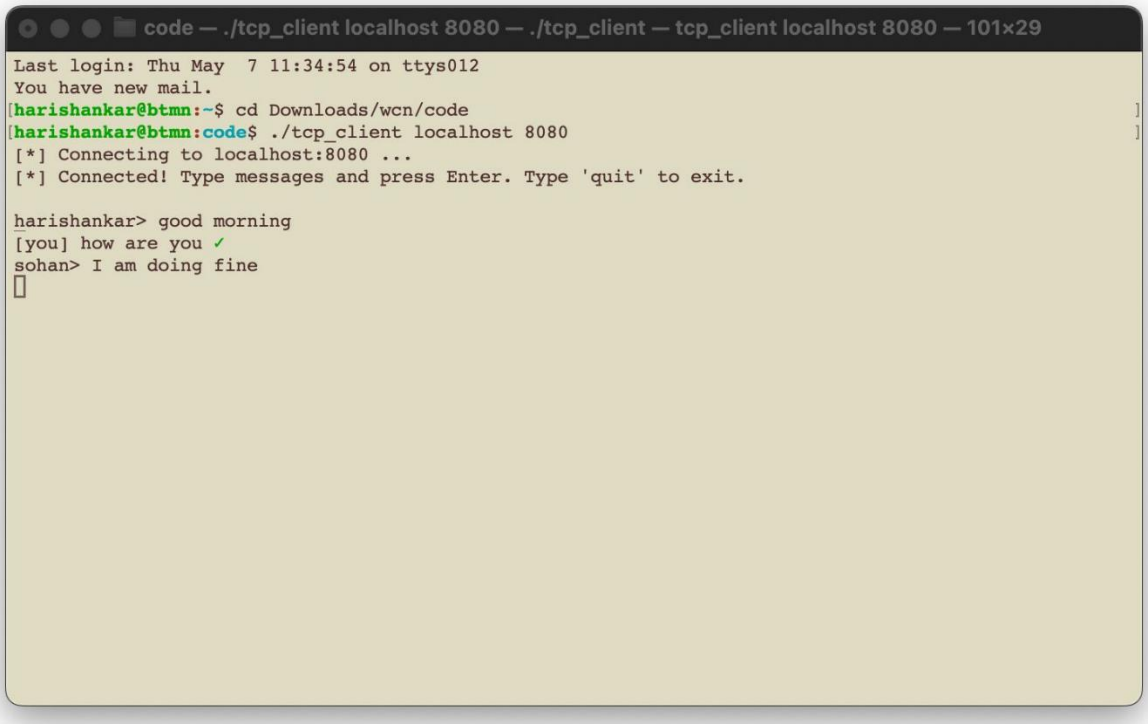
Server tracking clients :



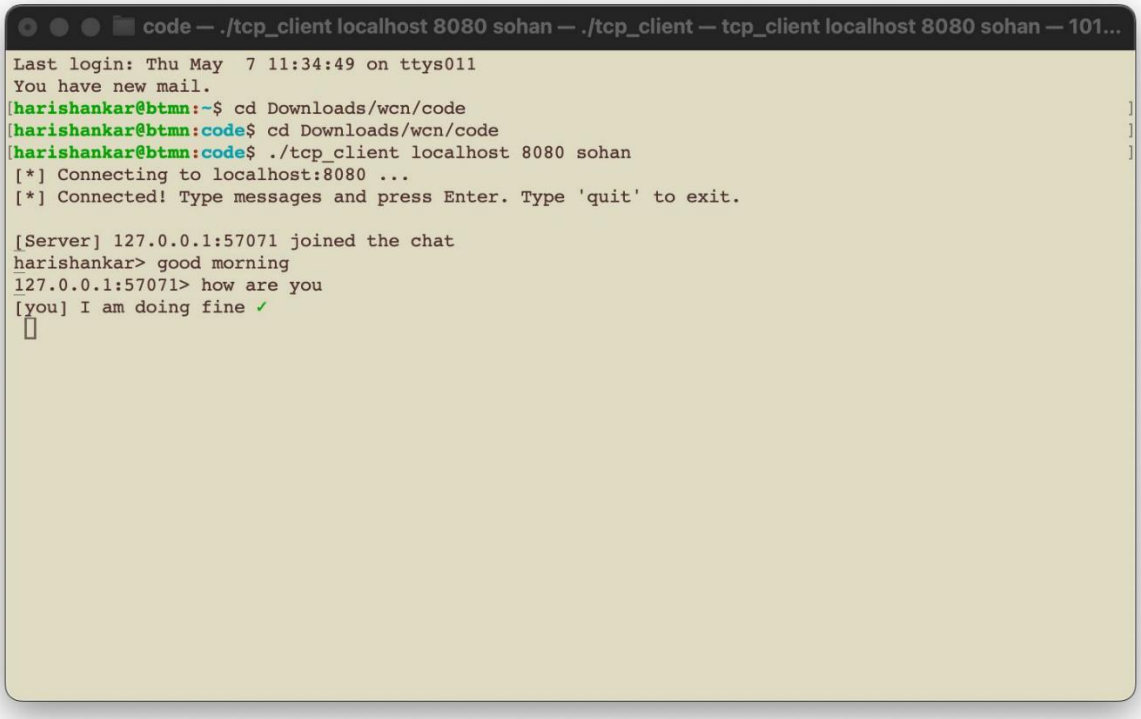
Client1 (Harishankar) messages:



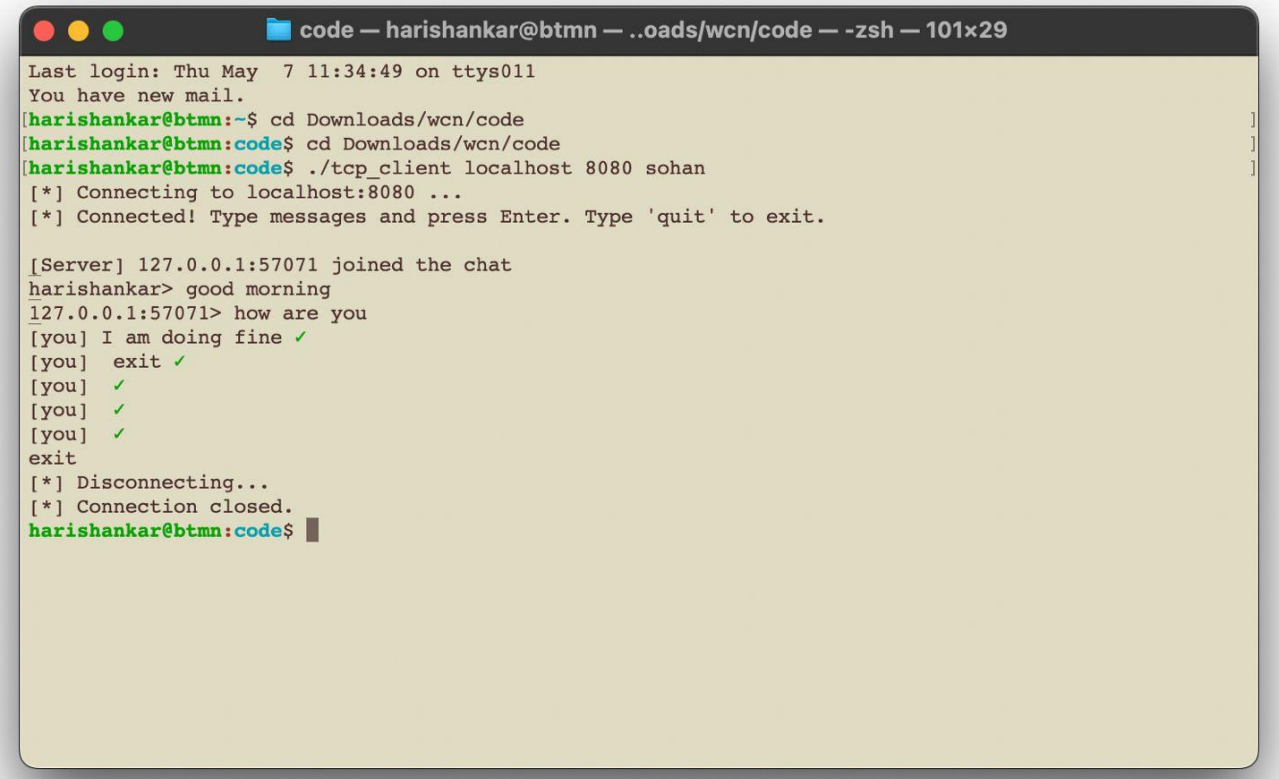
Client2 messages:

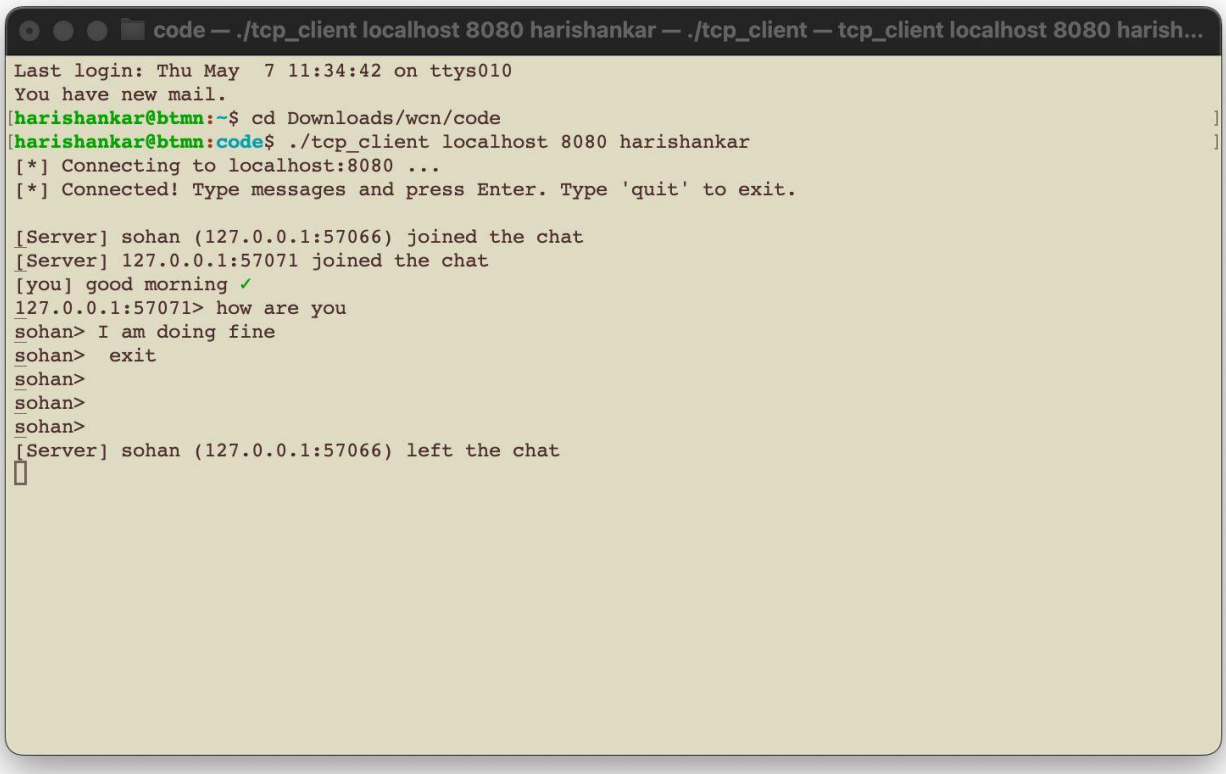


Client2 (Sohan) messages:

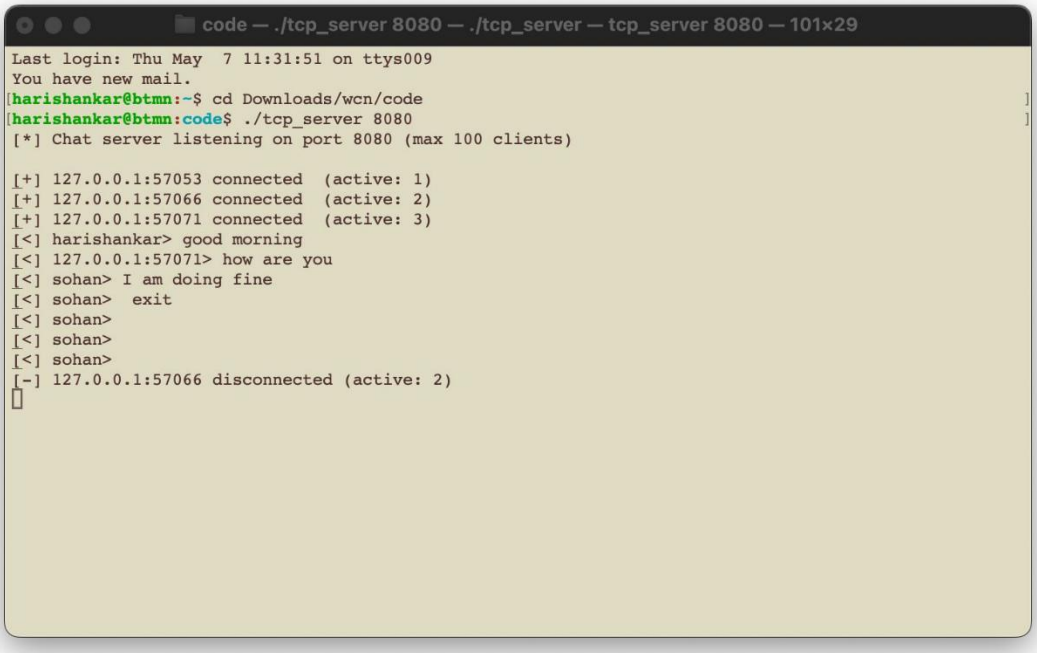


When client will send message “exit” to server:





How server will receive message when client sends “exit”:



Conclusion:

The TCP client–server program was successfully implemented using socket programming in C. A reliable connection was established between the client and server using the TCP protocol, ensuring accurate and ordered data transmission. The server was able to handle communication and broadcast messages to multiple clients effectively. The program demonstrated the key features of TCP such as connection establishment, reliable data transfer, and proper communication between systems. Hence, the objective of implementing TCP protocol for client–server communication was achieved successfully.

2)

Aim:

Extraction of RDS (Radio Data System) data from FM broadcasts using RTL SDR receiver. Frequency – 92.7 MHz:

Tool used : RTL-SDR dongle, Antenna, PC, GNU Radio Companion (GRC)

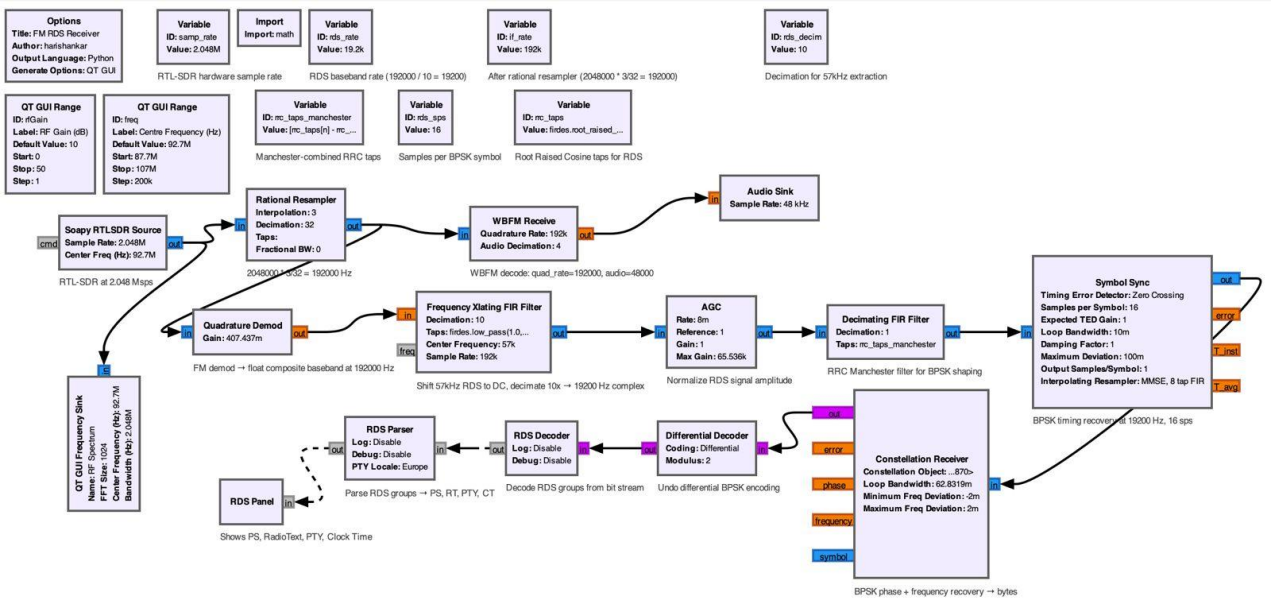
Theory:

FM broadcasting transmits audio information using frequency modulation along with additional multiplexed components such as a pilot tone, stereo difference signal, and the Radio Data System (RDS) subcarrier. RDS is a digital communication protocol embedded within the FM signal that uses binary phase shift keying to transmit metadata like station name, program type, and radio text. In a Software Defined Radio implementation using an RTL-SDR receiver, the received signal is first tuned and digitized, followed by demodulation to recover the baseband composite signal. The RDS component is then isolated using filtering, shifted to baseband, and processed through decimation and other filtering stages to improve signal quality. Finally, demodulation, clock recovery, and decoding techniques are applied to reconstruct the digital bitstream and extract meaningful RDS information, demonstrating the flexibility of SDR in real-time signal processing without dedicated hardware.

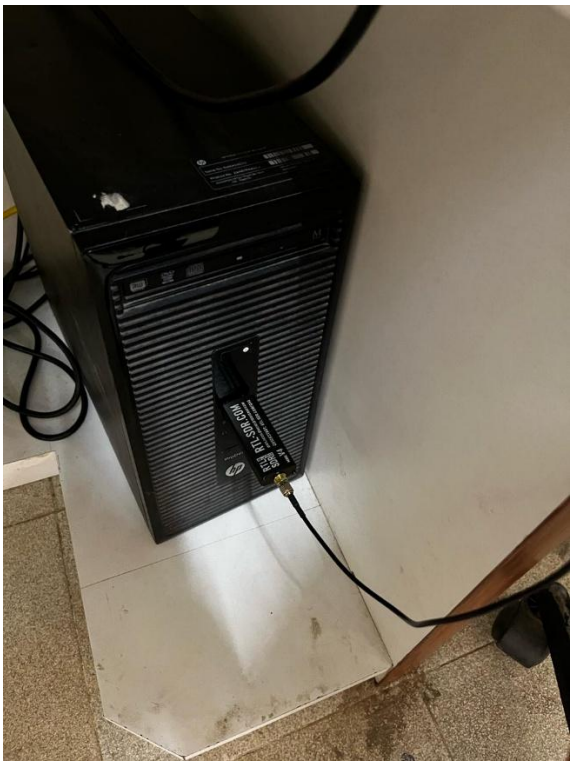
Procedure:

- Connect the RTL-SDR dongle and antenna to the system.
- Install drivers and verify the device is detected.
- Open GNU Radio Companion and create a new flowgraph.
- Add RTL-SDR source block and tune to the required FM station.
- Set appropriate sampling rate and RF gain.
- Apply low-pass filtering and perform FM demodulation.
- Use band-pass filtering to extract the RDS component.
- Shift the RDS signal to baseband and apply decimation.
- Perform BPSK demodulation with clock recovery.
- Decode RDS data and observe results using plots.

Flow chart :

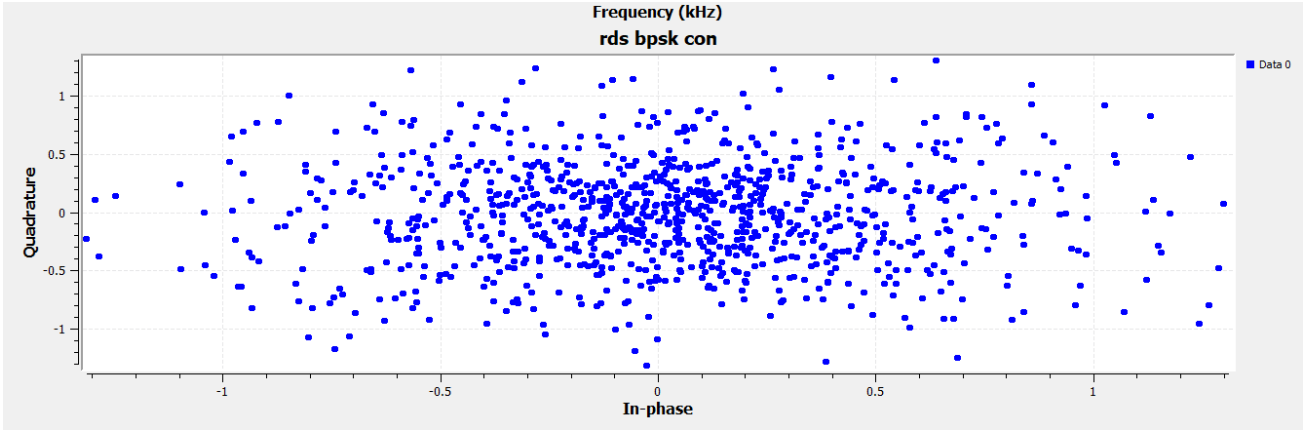


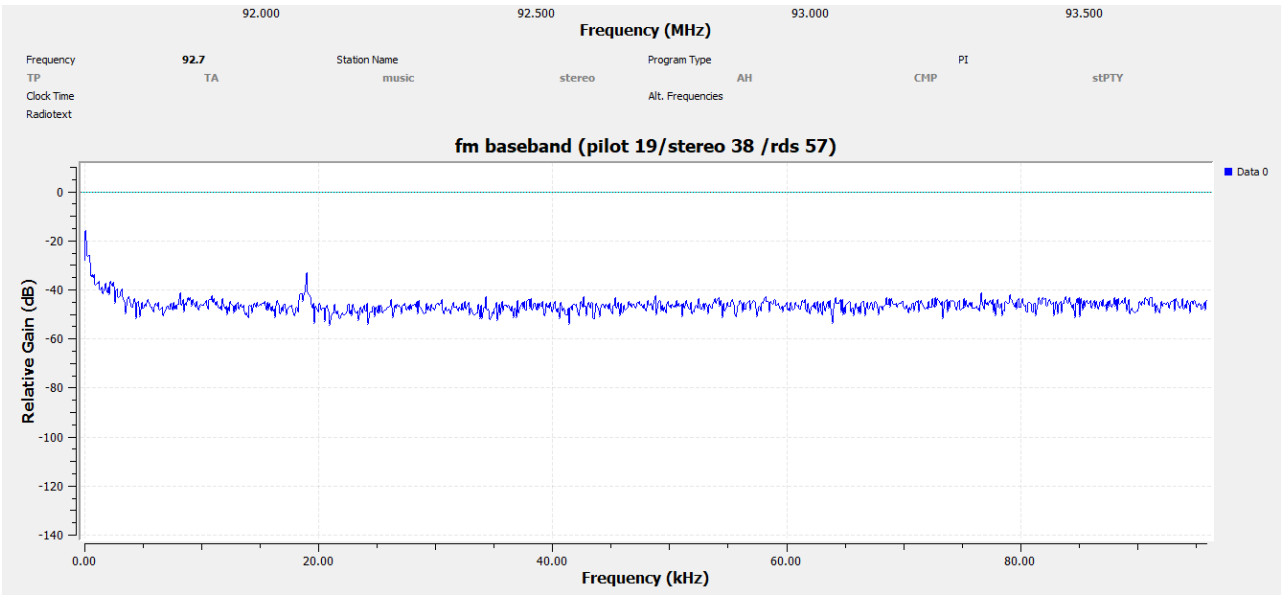
Hardware Connections :

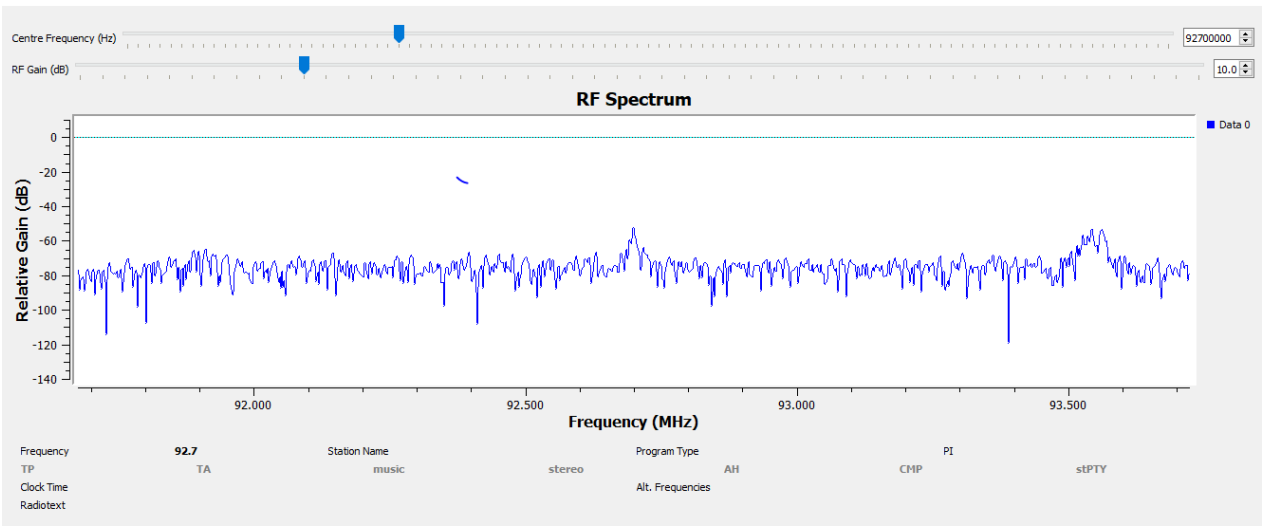




Results :







Conclusion :

The system successfully demonstrates the extraction of Radio Data System information from an FM broadcast at 94.3 MHz using an RTL-SDR receiver and software-defined radio techniques. By tuning to the required frequency, processing the received signal, isolating the RDS component, and decoding it, useful information such as station identification and program details is obtained.

This implementation highlights the effectiveness of SDR in real-time signal analysis and digital data extraction, offering a flexible and cost-efficient approach for modern wireless communication applications without relying on dedicated hardware.